SIMATS ENGINEERING

SCENARIO-BASED ASSESSMENT

SOFTWARE TESTING

Student Name	Bhavani Sankar
Register Number	192411238
Course Code	CSA3712
Assessment	CO2 – AT1 Scenario-Based Assessment
Course Outcome	Design effective test cases using black-box and white-box testing techniques and control methods flow.

Submitted by: Bhavani Sankar

Register Number: 192411238

ANSWERS TO SCENARIO-BASED QUESTIONS

The answers follow the 15 scenarios in the supplied assessment. Where exact business limits or source code are not provided, the test designs use symbolic limits and conceptual control-flow models; actual application specifications should determine final values.

1. Online Banking Fund Transfers: Boundary Value Analysis and Equivalence Partitioning

Scenario Analysis

Frequent failures near minimum and maximum transfer limits suggest that boundary conditions, validation rules, or amount calculations are not being tested systematically. Random testing may miss rare boundary values and does not guarantee repeatable coverage.

Test Design

Assume the business rule allows transfers from a minimum amount L to a maximum amount U, inclusive. Use the actual values configured by the bank rather than inventing limits. Test L–1, L, L+1, a normal value, U–1, U, and U+1. Also test zero, negative values, blank input, non-numeric text, excessive decimal places, and amounts exceeding the available balance.

Equivalence Classes

Valid class: numeric amounts within [L,U] and not greater than the available balance. Invalid classes: below L, above U, non-numeric or missing amount, invalid precision, insufficient balance, blocked account, and invalid beneficiary. Choose at least one representative from each class.

Optimized Approach

Combine ECP and BVA with decision-table tests for balance status, beneficiary status, daily limit, and authentication result. Verify successful debit and credit, transaction reference, rollback on failure, duplicate-submit prevention, and audit logs. Use repeatable test data and automate regression tests.

Decision and Justification

Random testing can remain as supplementary fuzz or exploratory testing, but it should not be the primary strategy. Structured boundary and equivalence tests are more reproducible and target the likely fault area. Acceptance requires correct outcomes at every boundary, no unauthorized transfer, and no partial debit/credit after a failed transaction.

2. Healthcare Application: White-Box Testing and Test Adequacy

Scenario Analysis

Critical defects after deployment indicate that the test suite did not adequately cover the module's statements, decisions, execution paths, or error-handling conditions. Passing a small number of normal cases is not evidence of sufficient testing.

Conceptual Control-Flow Graph

Example: Start → validate patient → [valid?]. If no, display validation error → End. If yes, read clinical data → [data complete?]. If no, record missing-data error → End. If yes, calculate result → [result within safe range?]. If no, raise alert and require review → End. If yes, save result → display confirmation → End. Each decision is a branch; each arrow represents possible control flow.

Coverage Evaluation

Statement coverage checks whether each executable statement runs at least once, but can miss an untested decision outcome. Branch coverage checks both true and false outcomes of each decision. Path coverage checks combinations of routes through the graph; complete path coverage may be infeasible when loops or many combinations exist. For healthcare software, branch coverage alone is not enough for high-risk logic.

Improved Strategy

Create tests for valid and invalid patient identifiers, missing and malformed clinical data, safe and unsafe results, storage failure, unavailable services, and recovery. Ensure every decision outcome is exercised. Identify independent paths using cyclomatic complexity, then add tests for critical combinations, exception handling, and loop boundaries. Use requirements traceability to link each safety requirement to tests.

Reliability Impact

Run automated unit tests on each change, integration tests for interfaces and databases, and risk-based end-to-end tests before release. Review coverage reports alongside defect severity and requirements coverage. This reduces untested routes and helps prevent unsafe results, lost records, and incorrect alerts. Coverage is evidence of execution, not proof that the software is defect-free.

3. Retail E-Commerce: Price, Quantity, and Discount Combinations

Problem Analysis

Testing each input separately may miss interaction defects, such as a valid price combined with an invalid quantity or a discount exceeding the order value. The current strategy is incomplete if it does not test rules across inputs.

ECP and BVA

For price, define valid positive prices and invalid zero, negative, blank, or non-numeric values. For quantity, test the minimum allowed, typical quantity, maximum allowed, zero, negative, and one above maximum. For discount, test zero, the allowed maximum, just below and above that maximum, negative discount, and discount greater than the subtotal. Use the actual business limits.

Decision Table

Create conditions for valid price, valid quantity, discount eligibility, discount within limit, and customer eligibility. Outcomes should include reject input, calculate without discount, apply discount, or reject discount. Include representative tests for every meaningful rule combination, especially combinations that could make the total negative or produce rounding errors.

Additional Techniques

Use pairwise/combinatorial testing when the full Cartesian product is too large, while retaining targeted tests for high-risk three-way interactions. Add boundary tests, negative tests, currency/rounding tests, and property checks such as total ≥ 0 and discounted total $\leq$ subtotal when the rules require it.

Expected Result

The revised suite combines equivalence classes, boundaries, decision tables, and risk-based combinations. Automate expected-total calculations independently and run regression tests after pricing changes. This gives better interaction coverage without executing every theoretically possible combination.

4. Complex Nested Conditions and Loops: Path Testing

Current Gaps

Partial path coverage may result from an incomplete CFG, overlooked branches in nested conditions, untested loop exits, or test cases that execute only typical inputs. Nested loops can create many paths, and loops may repeat zero, one, or many times.

CFG and Independent Paths

Draw nodes for statements or blocks, edges for control transfers, and decision nodes for each condition. Confirm that every if/else, switch outcome, loop entry, loop continuation, and loop exit is represented. Cyclomatic complexity can be calculated as $V(G)=E–N+2P$, where E is edges, N is nodes, and P is connected components (normally 1). It estimates the number of linearly independent paths.

Testing Strategy

Design a basis set of independent paths that exercises each decision outcome. For every loop, test zero iterations where permitted, one iteration, two iterations, typical iteration count, maximum supported count, and termination conditions. For nested loops, vary inner-loop entry and exit while testing outer-loop boundaries. Add invalid input and exception paths.

Coverage and Prioritization

Aim for 100% statement and branch coverage for the module where feasible, then use basis-path coverage and risk analysis for complex routes. Full path coverage may be impossible because loops can create an unbounded number of paths; document infeasible paths and justify the selected risk-based coverage.

Expected Outcome

CFG review and independent-path tests expose missing branches, incorrect loop termination, and state errors. Combine static analysis, code review, automated unit tests, and regression tests. Coverage reports should be checked against requirements and defect history rather than treated as the only quality measure.

5. Financial Application: Hybrid Black-Box and White-Box Testing

Why Defects Remain

Using both black-box and white-box testing does not guarantee quality if the tests are poorly designed, coverage is measured only by totals, requirements are missing, or integration and production-like conditions are ignored.

Black-Box Layer

Apply ECP and BVA to transaction amount, account balance, transfer limits, fees, dates, and customer status. Use decision tables for combinations such as sufficient/insufficient funds, active/frozen account, valid/invalid beneficiary, and daily-limit status. Check both expected outputs and side effects.

White-Box Layer

Build CFGs for authorization, validation, balance checking, debit, credit, and rollback. Test each decision outcome and independent path. Verify error handling, exception paths, transaction atomicity, and whether failures correctly roll back changes. Review statement, branch, and basis-path coverage for high-risk components.

Integrated Test Plan

Map requirements and risks to tests. Unit-test calculation and validation logic; integration-test databases, payment gateways, and messaging; run end-to-end tests for complete transactions; and conduct security, concurrency, recovery, and performance tests. Use synthetic test accounts and avoid real customer data.

Justification

Black-box tests show whether the system meets user-visible requirements, while white-box tests reveal unexecuted internal logic. Combining them with traceability, code review, automated CI regression, and defect analysis improves coverage and prevents recurrence. Release only when critical defects are resolved and agreed coverage and business acceptance criteria are met.

6. HR Payroll Bonuses at 5, 10, and 15 Years

Failure Analysis

Incorrect payouts at exactly 5, 10, and 15 years strongly suggest off-by-one errors, incorrect comparison operators, or overlapping service bands. Exploratory testing alone may not reliably test every threshold.

Boundary Value Analysis

For each threshold T in {5,10,15}, test $T-\varepsilon$, exactly T, and $T+\varepsilon$, where ε is the smallest meaningful service increment in the system. If service is stored in completed years, use the immediately preceding and following supported values; if stored in days or months, test just before, on, and just after the anniversary date.

Equivalence Classes

Partition employees into service bands defined by the official bonus policy, such as below 5 years, 5 to below 10, 10 to below 15, and 15 years or more, if those are the actual rules. Within each band, test representative employees. Partition performance ratings into every valid rating and invalid/missing values.

Combination Tests

Build a decision table crossing service band with performance rating. Verify bonus percentage, base salary, rounding, eligibility, and effective date. Include leap-day anniversaries, future hire dates, terminated employees, and changes in rating if these are supported by the specification.

Why Structured Testing Is Better

BVA targets the exact thresholds where the observed faults occur, and ECP reduces redundant tests while preserving class coverage. Exploratory testing remains useful for unexpected behavior but should supplement, not replace, repeatable threshold tests. Expected payouts must be independently calculated from the approved policy.

7. Laboratory Temperature Control: CFG and MC/DC

Safety Concern

A missing alarm during a sensor-malfunction sequence is a serious safety defect. Testing only normal temperature conditions or achieving basic branch coverage may not prove that each condition independently affects the alarm decision.

Conceptual Logic and CFG

Start → read temperature and sensor status → decision: sensor valid? If no, activate fault alarm and enter safe state. If yes, decision: temperature above high limit? If yes, activate cooling and high-temperature alarm. Otherwise, decision: temperature below low limit? If yes, activate heater. Otherwise maintain normal operation. Every decision outcome must have an edge in the CFG; add explicit paths for failed readings and actuator faults.

MC/DC Approach

List the Boolean conditions in each safety decision, such as sensor_valid, temperature_high, and temperature_low. For each condition, construct test pairs where only that condition changes while all other conditions remain fixed, and demonstrate that the overall decision outcome changes. Use the actual logic and requirements to create the MC/DC table; avoid contradictory high/low conditions.

Test Cases

Test valid sensor/normal range, valid sensor/high temperature, valid sensor/low temperature, invalid sensor, missing or out-of-range reading, recovery after sensor restoration, and simultaneous faults. Confirm alarm activation, safe-state behavior, event logging, and alarm reset rules.

Impact and Limits

Target 100% MC/DC for the identified safety decision logic, supported by branch coverage, hardware-in-the-loop tests, fault injection, and regression tests. MC/DC improves confidence that each condition independently influences decisions, but it does not replace hazard analysis, requirements validation, or independent safety review.

8. Travel Booking Promo Codes and Negative Totals

Problem Analysis

Testing promo codes one at a time misses interaction effects among promotion type, price, customer tier, stacking rules, and order of application. Invalid stacking can apply discounts twice or create a negative total.

Decision-Table Conditions

Use conditions such as price valid, promo A valid, promo B valid, stacking permitted, user tier eligible, and discount capped at the eligible subtotal. Outcomes include accept both codes, accept one code, reject an ineligible code, or reject the transaction. Include precedence and expiry rules if defined.

Representative Test Set

Test no promo, one valid fixed-value promo, one valid percentage promo, two compatible promos, two incompatible promos, expired promo, tier-ineligible promo, discount greater than subtotal, zero/negative price, and boundary prices around minimum-spend requirements. Test promo order if the business rules make order relevant.

Cause-Effect and Constraints

Map each input condition to effects such as discount eligibility, stacking decision, and final total. Apply constraints to exclude impossible combinations and use pairwise testing for lower-risk combinations. Keep targeted tests for high-risk interactions and assert that the total cannot be negative unless the specification explicitly permits credits.

Optimization

Decision tables make rule gaps visible and reduce duplicate cases. Automate calculation checks, logging, and regression tests. The final suite should cover every meaningful business rule and invalid combination, not necessarily every mathematically possible input combination.

9. Legacy Database Migration: Nested Loops and Infinite Loops

Why Statement Coverage Failed

High statement coverage only proves that statements ran; it does not prove loops terminate, conditions change correctly, or every loop exit is exercised. A test can execute the loop body once and still miss an infinite-loop defect.

CFG and Cyclomatic Complexity

Construct a CFG with nodes for initialization, loop conditions, record validation, nested-loop conditions, update operations, error handling, and exit. Calculate V(G)=E−N+2P. The result estimates the number of independent paths and helps organize a basis-path test set, but does not enumerate every possible iteration count.

Path and Loop Tests

Test empty input, one record, multiple records, malformed record, duplicate record, invalid field, inner-loop zero/one/multiple iterations, and maximum supported dataset. Verify that loop counters or iterators advance, termination conditions eventually become false, and errors do not skip required cleanup.

Stress and Recovery

Run large datasets, prime and non-prime sizes, boundary sizes, and randomized but seeded datasets. Add timeouts, watchdogs, memory monitoring, and assertions on processed-record counts. Inject database errors and confirm safe termination and resumability.

Conclusion

Basis-path testing and loop-boundary testing complement statement coverage. Use static analysis, code review, and performance/stress tests to identify non-terminating paths. Cyclomatic complexity helps prioritize independent paths, while requirements and data-volume testing validate real migration behavior.

10. Food Delivery Tracker: State Transition Testing

State Model

Model the tracker states as Ordered, Preparing, Out for Delivery, Delivered, Cancelled, and possibly Failed/Refund Pending where supported. Define allowed transitions explicitly; for example, Ordered → Preparing → Out for Delivery → Delivered. Cancellation rules depend on the current state and business policy.

Transition Coverage

Test every valid transition and every important invalid transition. Include cancellation before preparation, cancellation during preparation, driver cancellation while Out for Delivery, reassignment to another driver, customer cancellation after dispatch, delivery completion, network interruption, and duplicate status events.

Transition Pairs and Recovery

Test sequences of two or more transitions, especially Out for Delivery → Driver Cancelled → Reassign Driver → Out for Delivery. Verify that the app does not remain stuck, stale updates are rejected, and the customer receives accurate notifications. Test timeout and retry behavior when the server or network is unavailable.

Why ECP/BVA Are Insufficient

ECP and BVA focus mainly on input values and numeric boundaries. They may not reveal illegal state changes, missing recovery transitions, or event-order problems. State transition testing checks both the current state and the event that causes the next state.

Expected Outcome

Create a state-transition table and automate tests for states, transitions, transition pairs, and invalid events. Confirm that backend status, driver assignment, customer display, notifications, and refund logic remain consistent. This directly targets the reported cancellation failure.

11. Cybersecurity Authentication: Hybrid Testing Against Brute Force

Risk Analysis

Brute-force vulnerabilities can occur when repeated attempts are not limited, OTPs remain valid too long, errors reveal account existence, or IP/location checks can be bypassed. Happy-path login tests are insufficient.

Black-Box and Error Guessing

Test wrong passwords repeatedly, expired/reused/wrong OTPs, OTP replay, rapid attempts, account enumeration, missing fields, concurrent sessions, unusual IP changes, and recovery flows. Check rate limits, temporary lockouts, progressive delays, alerts, and generic error messages. Use only authorized test accounts and a controlled test environment.

White-Box Path Analysis

Build a CFG for password verification → attempt-count check → OTP generation/validation → IP/risk evaluation → grant or deny access. Test all true/false branches and independent paths, including lockout, OTP expiry, invalid IP, service failure, and account recovery. Inspect whether counters reset only under the correct conditions.

Security Controls

Verify server-side rate limiting by account and source, OTP expiry and one-time use, secure session handling, audit logs, monitoring, and safe recovery. Use threat modeling and automated security tests; test secrets and sensitive data are not logged. Avoid relying on IP location alone as an authentication factor.

Justification

Error guessing explores likely abuse cases, while path analysis identifies untested logic routes. Together with penetration testing by authorized personnel, code review, and regression tests, this hybrid strategy provides stronger evidence that authentication controls work under both normal and adversarial conditions.

12. E-Learning Subscriptions: Expiration-Day Upgrades

Problem Analysis

Errors on the expiration date may arise from inclusive/exclusive date comparisons, time-zone handling, or the order in which expiration and upgrade events are processed. Tests should define the policy for whether an upgrade on the expiry date is valid.

Equivalence Classes

Partition subscription timing into before expiration, exactly on expiration, after expiration, and invalid dates. Partition subscription status into active, expired, suspended, and cancelled where applicable. Partition plan types into Free, Basic, and Premium and test permitted upgrade/downgrade paths.

Boundary Tests

Test immediately before the expiration instant, exactly at it, immediately after it, midnight transitions, different time zones, month-end, leap day, and daylight-saving transitions if applicable. Use the application's documented time-zone and date policy.

Structured Test Design

Create a decision table crossing current plan, target plan, subscription status, and upgrade time. Verify access permissions, billing/proration, renewal date, course availability, confirmation messages, and duplicate-payment prevention. Test failed payment and retry scenarios.

Completeness

ECP gives representative coverage of date and plan classes; BVA targets the exact expiration boundary; state/transition tests check plan changes. Automate date-controlled tests with a fixed clock. These techniques reduce gaps and ensure the same policy is applied consistently across UI, API, and billing services.

13. Smart-Home Lighting: Branch and Condition Coverage

Risk Analysis

Statement coverage may execute each statement without testing every switch outcome or Boolean condition. Incorrect combinations can cause lights to become too bright, too dim, or fail to respond to sensor readings.

CFG Construction

Create nodes for sensor read, sensor validity, ambient-light range selection, manual override, dimming calculation, and hardware command. Add separate branches for each switch case, default case, and Boolean decision. Include sensor errors and actuator communication failures.

Branch and Condition Tests

For each decision, test true and false outcomes of each condition. For a compound expression such as sensor_valid AND manual_override_off, create tests that independently vary each condition. Test all switch cases, boundary readings around each light-level threshold, and unknown/default sensor values.

Integration Strategy

Use unit tests for the decision logic, simulation for sensor inputs, and hardware-in-the-loop tests for actual dimmer response. Verify correct output levels, safe fallback on sensor failure, and recovery after communication loss. Test simultaneous changes in ambient light and manual override.

Justification

Branch coverage exposes unexecuted decision outcomes, while condition coverage checks individual Boolean components. Combining both with boundary tests, integration tests, and regression automation reduces the chance that software logic and hardware behavior disagree.

14. Logistics Shipping Costs: Robust Boundary Value Analysis

Failure Analysis

Missing zero and negative weights indicates that validation tests did not include invalid boundaries. The system may calculate a price for physically impossible input or accept malformed data.

Robust BVA

If the minimum valid weight is Wmin and maximum is Wmax, test Wmin–ε, Wmin, Wmin+ε, a typical value, Wmax–ε, Wmax, and Wmax+ε. Include zero, negative values, blank input, non-numeric input, excessive precision, and extremely large values. Use the actual specified limits and smallest accepted unit.

Other Dimensions

For destination zone and delivery speed, test every valid category and invalid/unknown values. Use decision tables to check how weight bands, destination, and speed combine to determine the shipping charge. Include the exact transition between weight brackets.

Expected Behavior

Invalid weights should be rejected with a clear message and must not create an order or charge. Valid weights should produce the documented price, with correct rounding and currency. Test API validation as well as the user interface so that invalid data cannot bypass client-side checks.

Justification

Robust BVA deliberately includes values just outside valid boundaries. Negative tests are necessary because users, integrations, or malicious requests can send invalid data. They improve input validation, prevent incorrect billing, and strengthen reliability.

15. Medical Imaging: Basis Path Testing and Prime Data Sizes

Problem Analysis

Inconsistent results only for prime-sized datasets may indicate incorrect assumptions about chunking, partitioning, thread allocation, remainder handling, or synchronization. Random testing may rarely select prime sizes or repeat the exact execution conditions.

White-Box Design

Inspect the parallel loops and build a CFG for data initialization, partitioning, worker execution, remainder processing, result merging, and error handling. Calculate cyclomatic complexity V(G)=E−N+2P and design a basis set of independent paths. Review shared variables, race conditions, and loop bounds.

Structured Test Data

Test dataset sizes 0 where permitted, 1, small prime sizes, nearby composite sizes, sizes just below/at/above worker-count multiples, and the maximum supported size. Include primes near chunk boundaries and datasets with known expected outputs. Repeat runs to detect nondeterministic results.

Accuracy and Stability

Compare parallel output with a trusted serial reference implementation using tolerances defined for the imaging algorithm. Verify that every element is processed exactly once, no data is skipped or duplicated, and results do not depend on thread scheduling. Test worker failure and cancellation where supported.

Justification

Basis-path testing targets independent control-flow routes; loop-boundary and prime/composite data tests target partitioning edge cases. Combine these with race detection, stress testing, deterministic seeds, and regression tests. This provides stronger evidence of numerical correctness and system stability than unstructured random testing alone.